LOS + LMS Prototype - Low-Level Design

LMS Prototype

Author Mobasher Ali
Date 2026-08-05
Contact mobas@example.com
Repository https://github.com/mobas

LOS + LMS Prototype — Low-Level Design (LLD)

Date: 2026-08-05 **Companion docs:** Spec · HLD · Plan

1. Module map

Every file in the repo, with its responsibility and the symbols it exports. Tasks in the plan add to this list one at a time.

`packages/shared/src/index.ts`

- **Responsibility:** Single source of truth for cross-tier types and validation.
- **Exports:**
  - `LoanStatus = 'pending' | 'approved' | 'rejected' | 'disbursed'`
  - `LoanLifecycle = 'active' | 'closed'`
  - `Employment = 'salaried' | 'self_employed'`
  - `Role = 'applicant' | 'admin'`
  - `SignupInput` (Zod), `LoginInput` (Zod)
  - `ApplicationInput` (Zod): `{ amount: number (positive, ₹1k–₹10L), termMonths: enum [6,12,24,36], annualRateBps: enum [1000,1200,1500,1800,2000,2400], purpose: string 1..200, employment: Employment }`
  - `DecisionInput` (Zod): `{ decision: 'approve' | 'reject', reason?: string 1..500 }`
  - DTOs (inferred TS types): `UserDTO`, `ApplicationDTO`, `LoanDTO`, `InstallmentDTO`

`apps/api/src/`

File	Responsibility	Key exports
`server.ts`	Process entry. Loads env, builds app, calls `migrate()` + `seed()` then listens.	(default export) `start()`
`app.ts`	Fastify factory. Registers plugins, decorators, error handler, static handler.	`buildApp()`
`db/client.ts`	Initializes better-sqlite3 + Drizzle. Reads `DATABASE_URL`.	`db` (Drizzle instance), `sqlite` (raw handle)
`db/schema.ts`	Drizzle table definitions. See §2.	All table objects + inferred row types
`db/migrate.ts`	Applies Drizzle migrations. Idempotent.	`migrate()`

File	Responsibility	Key exports
`db/seed.ts`	Idempotent upsert of two applicants + one mid-flight loan. See §6.	`seed()`
`domain/amortization.ts`	Pure functions: `calculateEmi(principal, annualRateBps, termMonths)`, `generateSchedule(principal, annualRateBps, termMonths, startDate)`. See §4.	`calculateEmi`, `generateSchedule`
`domain/underwriting.ts`	Pure function: `evaluate(monthlyIncomeCents, principal, annualRateBps, termMonths)` returns `{ recommendation: 'approve' | 'reject', reason?: string }`. See §5.	`evaluate`
`lib/auth.ts`	`signToken(userId)`, `verifyToken(token)`, `hashPassword(pw)`, `verifyPassword(pw, hash)`, cookie helpers.	same
`lib/errors.ts`	Typed domain error classes + Fastify error mapper. See §7.	`AppError`, `ValidationError`, `UnauthorizedError`, `ForbiddenError`, `NotFoundError`, `ConflictError`, `errorHandler`
`lib/money.ts`	Helpers for integer-cents math. `roundCents(n)`, `formatINR(cents)`.	same
`routes/auth.ts`	`POST /signup`, `POST /login`, `POST /logout`, `GET /me`.	Fastify plugin
`routes/applications.ts`	`POST /applications` (creates `pending`), `GET /applications`, `GET /applications/:id`.	Fastify plugin

File	Responsibility	Key exports
`routes/loans.ts`	`GET /loans`, `GET /loans/:id`, `POST /loans/:id/installments/:n/pay`.	Fastify plugin
`routes/admin.ts`	`GET /admin/applications`, `POST /admin/applications/:id/decision`, `POST /admin/applications/:id/disburse`.	Fastify plugin

`apps/web/src/`

File	Responsibility	Key exports
`main.tsx`	Mounts React app, wraps in `QueryClientProvider` + `BrowserRouter`.	(entry)
`App.tsx`	Top-level routes + `<Toaster />`.	default
`lib/api.ts`	`apiFetch(path, opts)` — same-origin fetch with `credentials: 'include'`, throws on non-2xx.	`apiFetch`
`lib/format.ts`	`formatINR(cents)`, `formatDate(iso)`.	same
`hooks/useMe.ts`	TanStack Query: `['me']`.	`useMe`
`hooks/useApplications.ts`	TanStack Query: `['applications']`.	`useApplications`, `useCreateApplication`
`hooks/useLoans.ts`	TanStack Query: `['loans']` and `['loans', id]`.	`useLoans`, `useLoan`, `usePayInstallment`
`hooks/useAdmin.ts`	TanStack Query: `['admin','applications',status]`.	`useAdminApplications`, `useDecideApplication`, `useDisburse`
`components/AuthGuard.tsx`	Redirects to `/login` if no `me`; to `/dashboard` if `me.role !== 'admin'` on `/admin/*`.	default
`components/Money.tsx`	Renders `formatINR(value)`.	default

File	Responsibility	Key exports
`components/StatusBadge.tsx`	Maps `LoanStatus` / `LoanLifecycle` to shadcn `<Badge>` variants.	default
`routes/landing.tsx`	Hero + login/signup CTAs.	default
`routes/login.tsx`	RHF + Zod form. On success, navigate `/dashboard`.	default
`routes/signup.tsx`	RHF + Zod form, requires `monthlyIncome`.	default
`routes/apply.tsx`	3-step form (shadcn `Form` + simple stepper), submit creates `ApplicationDTO`.	default
`routes/dashboard.tsx`	Lists `applications` and active `loans`, "Next EMI" card per loan.	default
`routes/loan-detail.tsx`	Renders installments table, "Pay EMI" button on next unpaid row.	default
`routes/admin/queue.tsx`	Lists `pending` applications, links to detail.	default
`routes/admin/application-detail.tsx`	Renders application + rule recommendation + Approve/Reject form + (if approved) Disburse button.	default

2. Database schema (Drizzle, SQLite)

All money: `integer` (cents). All timestamps/dates: `text` (ISO 8601). All IDs: `text` (nanoid, 21 chars).

```
// apps/api/src/db/schema.ts
import { sqliteTable, text, integer, uniqueIndex } from 'drizzle-orm/sqlite-core';

export const users = sqliteTable('users', {
  id:             text('id').primaryKey(),
  email:          text('email').notNull().unique(),
  passwordHash:   text('password_hash').notNull(),
  fullName:       text('full_name').notNull(),
```

```
  role:           text('role', { enum: ['applicant', 'admin'] }).notNull(),
  monthlyIncome:  integer('monthly_income').notNull(),   // cents
  createdAt:      text('created_at').notNull(),
});

export const loanApplications = sqliteTable('loan_applications', {
  id:             text('id').primaryKey(),
  userId:         text('user_id').notNull().references(() => users.id),
  amountCents:    integer('amount_cents').notNull(),
  termMonths:     integer('term_months').notNull(),
  annualRateBps:  integer('annual_rate_bps').notNull(),
  purpose:        text('purpose').notNull(),
  employment:     text('employment', { enum: ['salaried', 'self_employed'] }).notNull(),
  status:         text('status', { enum: ['pending', 'approved', 'rejected', 'disbursed']
  decisionReason: text('decision_reason'),
  decidedBy:      text('decided_by').references(() => users.id),
  decidedAt:      text('decided_at'),
  disbursedAt:    text('disbursed_at'),
  createdAt:      text('created_at').notNull(),
});

export const loans = sqliteTable('loans', {
  id:               text('id').primaryKey(),                                  // = loanApp
  applicationId:    text('application_id').notNull().unique().references(() => loanApplica
  userId:           text('user_id').notNull().references(() => users.id),
  principalCents:   integer('principal_cents').notNull(),
  annualRateBps:    integer('annual_rate_bps').notNull(),
  termMonths:       integer('term_months').notNull(),
  startDate:        text('start_date').notNull(),
  endDate:          text('end_date').notNull(),
  status:           text('status', { enum: ['active', 'closed'] }).notNull(),
  outstandingCents: integer('outstanding_cents').notNull(),
});

export const installments = sqliteTable('installments', {
  id:           text('id').primaryKey(),
  loanId:       text('loan_id').notNull().references(() => loans.id),
  sequence:     integer('sequence').notNull(),
  dueDate:      text('due_date').notNull(),
  principalDue: integer('principal_due').notNull(),
  interestDue:  integer('interest_due').notNull(),
  paidAmount:   integer('paid_amount').notNull().default(0),
  paidAt:       text('paid_at'),
}, (t) => ({
  uniqLoanSeq: uniqueIndex('uniq_loan_seq').on(t.loanId, t.sequence),
}));
```

Inferred row types (used everywhere downstream):

```
type User              = typeof users.$inferSelect;
type NewUser           = typeof users.$inferInsert;
type LoanApplication   = typeof loanApplications.$inferSelect;
type Loan              = typeof loans.$inferSelect;
type Installment       = typeof installments.$inferSelect;
```

3. API contracts

All routes return JSON. Errors: `{ error: string, message?: string, issues?: ZodIssue[] }`. Auth via cookie `lms_session` (httpOnly, SameSite=Lax). `401` no/invalid session, `403` wrong role, `404` not found / not owner, `409` domain conflict, `400` validation.

`POST /api/auth/signup`

- Body: `SignupInput = { email, password (>=8), fullName, monthlyIncome (>=0) }`
- 201: `{ user: UserDTO }` + sets cookie
- 409: `{ error: 'EmailAlreadyUsed' }` if email exists

`POST /api/auth/login`

- Body: `LoginInput = { email, password }`
- 200: `{ user: UserDTO }` + sets cookie
- 401: `{ error: 'InvalidCredentials' }`

`POST /api/auth/logout`

- 204, clears cookie

`GET /api/auth/me`

- 200: `{ user: UserDTO }`
- 401: `{ error: 'Unauthorized' }`

`POST /api/applications`

- Auth: any session.
- Body: `ApplicationInput`.
- Behavior: compute `evaluate(monthlyIncome, amount, rate, term)`; if reject and `recommendation === 'reject'`, auto-set `status='rejected'`, `decision_reason = rule reason`, `decided_by = null`, `decided_at = now`. Else `status='pending'`. Returns the application with `recommendation` field.
- 201: `{ application: ApplicationDTO & { recommendation: { rule: 'approve' | 'reject', reason?: string } } }`

`GET /api/applications`

- Auth: any session.
- 200: `{ applications: ApplicationDTO[] }` (current user's only)

`GET /api/applications/:id`

- Auth: owner or admin.
- 200: `{ application: ApplicationDTO & { recommendation } }`
- 404: not found / not owner (and not admin)

`GET /api/loans`

- Auth: any session. Applicants see own; admins see all.
- 200: `{ loans: LoanDTO[] }`

`GET /api/loans/:id`

- Auth: owner or admin.
- 200: `{ loan: LoanDTO, installments: InstallmentDTO[] }`

`POST /api/loans/:id/installments/:n/pay`

- Auth: owner.
- Behavior: in a transaction, find installment with `sequence = :n` and `paid_at IS NULL`; if not found, rollback and 409. Else mark paid, decrement `loans.outstanding_cents`, if last installment set `loans.status='closed'`.
- 200: `{ installment: InstallmentDTO, loan: LoanDTO }`
- 409: `{ error: 'NoInstallmentFound' }` or `{ error: 'LoanClosed' }`

`GET /api/admin/applications?status=pending`

- Auth: admin.
- 200: `{ applications: (ApplicationDTO & { applicant: UserDTO, recommendation })[] }`

`POST /api/admin/applications/:id/decision`

- Auth: admin.
- Body: `DecisionInput`.
- Behavior: 404 if not found or not `pending`. 409 if not `pending`. UPDATE `status`, `decision_reason`, `decided_by = current user`, `decided_at = now`.
- 200: `{ application }`

`POST /api/admin/applications/:id/disburse`

- Auth: admin.
- Behavior: in a transaction, 404 if not found, 409 if not `approved`. Generate `startDate = today`, `endDate = startDate + termMonths`. Compute `outstandingCents = principalCents`. INSERT `loans` (id = application.id). Run `generateSchedule` and bulk INSERT `installments`. UPDATE `loanApplications.status='disbursed', disbursed_at=now`.
- 200: `{ loan, installments }`

4. Amortization (`apps/api/src/domain/amortization.ts`)

```
export function calculateEmi(
  principalCents: number,
  annualRateBps: number,
  termMonths: number
): number;   // integer cents, Math.round half-up

export interface ScheduleRow {
  sequence: number;          // 1..N
  dueDate: string;           // ISO yyyy-mm-dd, first = startDate + 1 month
  principalDue: number;      // integer cents
  interestDue: number;       // integer cents
}

export function generateSchedule(
  principalCents: number,
  annualRateBps: number,
  termMonths: number,
  startDate: string          // ISO yyyy-mm-dd
): ScheduleRow[];            // length = termMonths, sum(principalDue) === principalCents
```

Algorithm:

1. `r = annualRateBps / 10000 / 12` (monthly rate, decimal).
2. `emi = roundCents(principalCents * r * (1+r)^N / ((1+r)^N - 1))`. Special case `r === 0`: `emi = roundCents(principalCents / N)`.
3. For `i = 1..N-1`: `interestDue = roundCents(outstanding * r)`, `principalDue = emi - interestDue`, `outstanding -= principalDue`.
4. Final installment (`i = N`): `interestDue = roundCents(outstanding * r)`, `principalDue = outstanding` (forces the sum to match).
5. `dueDate` = add `i` months to `startDate` (using `date-fns/addMonths` with end-of-month clamp).

Worked example (₹50,000, 15% APR, 12 months, start 2026-01-01):

- `r = 0.0125`, `N = 12`, `emi ≈ 4,508.35` cents = ₹4,508.35.

- Final installment absorbs ~₹0.07 rounding.
- Schedule sums exactly to ₹50,000.00.

5. Underwriting (`apps/api/src/domain/underwriting.ts`)

```
export interface UnderwritingResult {
  recommendation: 'approve' | 'reject';
  reason?: string;            // present iff reject
}

export function evaluate(
  monthlyIncomeCents: number,
  principalCents: number,
  annualRateBps: number,
  termMonths: number
): UnderwritingResult;
```

Rules (in order, first match wins):

1. If `monthlyIncomeCents <= 0` → reject `"Income not provided"`.
2. Compute `emi = calculateEmi(...)`.
3. If `emi / monthlyIncomeCents > 0.50` → reject `"EMI exceeds 50% of income"`.
4. If `monthlyIncomeCents < 3 * emi` → reject `"Income insufficient for requested EMI"`.
5. Otherwise approve.

The application is **always** stored (either as `pending` or `rejected`). When the rule recommends reject, the application is auto-rejected with `decided_by = null` so the admin queue only shows genuinely pending work. The admin can still see auto-rejected applications on the applicant side via `GET /api/applications`.

6. State machine

Implemented in `routes/admin.ts` via a single `transition(app, target)` helper. Allowed transitions:

From	To	Trigger	Side effects
(none)	`pending`	`POST /applications` (rule ok)	INSERT row
(none)	`rejected`	`POST /applications` (rule no)	INSERT row, set `decision_reason`, `decided_at`

From	To	Trigger	Side effects
`pending`	`approved`	admin decision `approve`	set `decided_by`, `decided_at`
`pending`	`rejected`	admin decision `reject`	set `decided_by`, `decided_at`, `decision_reason`
`approved`	`disbursed`	admin disburse	INSERT `loans`, bulk INSERT `installments`, set `disbursed_at`

Rejected applications are terminal. Disbursed applications are terminal at the application level; lifecycle continues on the `loans` row.

7. Error model

```
// apps/api/src/lib/errors.ts
export class AppError extends Error { abstract readonly status: number; readonly code: stri
export class ValidationError extends AppError { status = 400; code = 'ValidationError'; }
export class UnauthorizedError extends AppError { status = 401; code = 'Unauthorized'; }
export class ForbiddenError extends AppError { status = 403; code = 'Forbidden'; }
export class NotFoundError extends AppError { status = 404; code = 'NotFound'; }
export class ConflictError extends AppError { status = 409; code: string; }   // subclass s

export function errorHandler(err, req, reply) {
  if (err instanceof ZodError) return reply.code(400).send({ error: 'ValidationError', issu
  if (err instanceof AppError) return reply.code(err.status).send({ error: err.code, messag
  req.log.error(err);
  return reply.code(500).send({ error: 'InternalError' });
}
```

Specific conflict codes: `EmailAlreadyUsed`, `InvalidStateTransition`, `NoInstallmentFound`, `LoanClosed`.

8. Test plan

Run with `node --test --import tsx apps/api/test/*.test.ts`. No test framework.

File	Coverage
`amortization.test.ts`	EMI matches closed-form for known inputs; schedule sums to principal; final absorbs rounding; `r=0` case; `addMonths` end-of-month clamp

File	Coverage
`underwriting.test.ts`	Each rejection rule triggers; boundary cases (FOIR exactly 0.50, income exactly 3× EMI); `income=0` rejected
`auth.test.ts`	Signup creates user + sets cookie; login round-trips; logout clears; `/me` 401 without cookie; duplicate email 409
`applications.test.ts`	Auto-rejected application returned to client; pending visible to admin; admin decision transitions state; disburse generates correct schedule and sets `outstanding_cents = principal`; double-disburse 409
`payment.test.ts`	Pay next unpaid installment decrements outstanding; mark last → `loans.status='closed'`; pay already-paid 409 (`NoInstallmentFound`); pay after close 409 (`LoanClosed`)

Tests use `':memory:'` SQLite, no network, no seed file. Web: no tests.

9. Seed data

`db/seed.ts` upserts deterministically (idempotent on email):

Email	Role	monthlyIncome (cents)	Loan
`admin@lms.dev`	admin	0	—
`alice@lms.dev`	applicant	5,000,000 (₹50,000)	none — fresh applicant
`bob@lms.dev`	applicant	10,000,000 (₹100,000)	5,000,000 cents / 12 mo / 15% APR, started 2026-05-01, 3 installments paid, 9 remaining

Passwords: `admin123`, `alice123`, `bob123` (bcrypt-hashed on seed). `bob`'s paid installments: `paid_at = (startDate + i months)`, `paid_amount = principalDue + interestDue`.

10. Money & formatting

- **Boundary:** integer cents only, both directions.
- **Display:** `formatINR(cents) = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' }).format(cents / 100)`.

- **Dates:** ISO `yyyy-mm-dd` for date-only fields, ISO 8601 with `Z` for timestamps. Client: `Intl.DateTimeFormat`.

End of LLD. Phased implementation tasks live in the Plan.